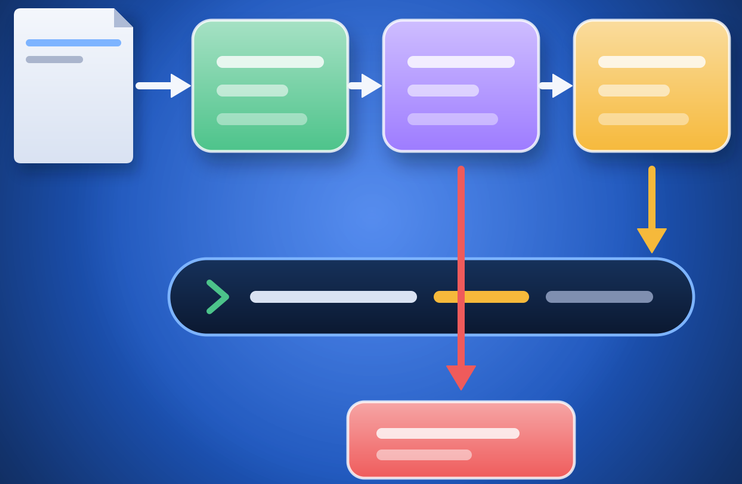
Institute of Technology of Cambodia
Faculty of Electrical Engineering
Génie Informatique et Communication

GICI4OS · OPERATING SYSTEMS · LAB 4

Redirection, Pipelines and Processes

Lab 4 guide: send output and errors to different places, connect commands, and build a report you can trust.



Heng Rathpisey · Department GIC · Academic year 2025–2026

Learning objectives

1. Use redirection and pipelines, and tell stdout, stderr and the exit status apart.
2. Filter and aggregate a CSV file, checking each stage of the pipeline.
3. Find out why a selection or a total is wrong, using normal and changed inputs.

THIS LAB

Lab 4

First the concepts, with examples. Then the lab on the server, where you write the script yourself. Watching and signalling processes are optional extensions, so they come at the end.

DISCUSSION

A command prints an error and a result. Where does each go?

Both appear on your screen. Are they the same thing? How would you save only one of them?

CONCEPTS FIRST

Streams, pipes and filters

Three ideas: every program has three streams, a pipe joins two programs, and small filters do one job each.

Every command has three streams

Stream	Number	Default	Used for
stdin	0	The keyboard	Input
stdout	1	The terminal	Normal output
stderr	2	The terminal	Error messages and diagnostics

Both output streams appear on the screen, so they look the same. They are different streams, and you can send them to different places.

The redirection operators

Operator	Meaning	Example
`>`	Send stdout to a file, replacing it	`ls > files.txt`
`>>`	Append stdout to a file	`echo done >> log.txt`
`2>`	Send stderr to a file	`ls /fake 2> errors.txt`
`2>&1`	Make stderr go where stdout goes now	`cmd > all.txt 2>&1`
`&>`	Bash shorthand: stdout and stderr into one file	`cmd &> all.txt`
`<`	Read stdin from a file	`sort < unsorted.txt`
`/dev/null`	Throw the output away	`cmd > /dev/null 2>&1`

A single > replaces the file. Use >> to add to it. In a pipeline, |& sends both streams to the next command.

Two streams, two files

student@server: ~/oslab-work/lab4

```
$ printf 'normal output\n' > evidence/stdout-demo.txt
$ cat missing-demo.txt 2> evidence/stderr-demo.txt
$ cat evidence/stdout-demo.txt
normal output
$ cat evidence/stderr-demo.txt
cat: missing-demo.txt: No such file or directory
```

printf writes the text. > sends it into the first file. The second command fails on purpose, and 2> catches only its error message.

Nothing about the error appeared on the screen the first time. It went into the file.

The order of redirections matters

stderr follows stdout to the file

student@server: ~/oslab-work/lab4

```
$ bash -c 'echo normal; echo problem >&2' > both.txt 2>&1
$ cat both.txt
normal
problem
```

First stdout goes to the file. Then 2>&1 points stderr at that same place.

stderr stays on the screen

student@server: ~/oslab-work/lab4

```
$ bash -c 'echo normal; echo problem >&2' 2>&1 > only-out.txt
problem
$ cat only-out.txt
normal
```

Here stderr is pointed at stdout while stdout is still the screen. Only then does stdout move to the file.

Read redirections from left to right.

A pipe joins two commands

- The pipe | connects the stdout of one command to the stdin of the next.
- The commands run at the same time. Data flows through as it is produced.
- Only stdout goes through the pipe. stderr still goes to the terminal.
- The exit status of a pipeline is, by default, the status of the last command.



Read it stage by stage: all rows, then only the ok rows, then only field 2 of those rows.

Small filters, one job each

Command	What it does	Example
grep	Keeps the lines that match a pattern	`grep '^ok,' events.csv`
cut	Takes fields from each line	`cut -d, -f2`
awk	Fields, conditions and arithmetic	`awk '{ total += $1 } END { print total+0 }'`
sort, uniq -c	Sort lines, then count equal neighbours	`sort data | uniq -c`
wc -l	Counts lines	`wc -l file.txt`
head, tail	The first or last lines	`head -n 1`
tee	Saves a copy and passes the data on	`ls | tee files.txt | wc -l`

uniq compares neighbours only, so sort first. tee lets you look inside a pipeline.

grep, cut and awk

grep: keep matching lines

- grep -v: lines that do not match.
- grep -c: only the count of matching lines.
- grep -i: ignore upper and lower case.
- grep -E: extended patterns.

cut: simple fields

- -d sets the delimiter, -f picks the fields.
- cut -d: -f1 /etc/passwd lists account names.
- It cannot do arithmetic or conditions.

awk: fields with logic

- $1 is the first field, $2 the second.
- END runs once, after the last line.
- awk -F: '$3 >= 1000' /etc/passwd keeps ordinary accounts.

Use cut for simple delimited fields. Use awk when you need a condition or a sum.

Rows are not counts

student@server: ~/oslab-work/lab4

```
$ printf 'ok,3\nfail,2\nok,5\n' | grep '^ok,' | wc -l
2
$ printf 'ok,3\nfail,2\nok,5\n' | grep '^ok,' | cut -d, -f2 | awk '{ t += $1 } END { print t+0 }'
8
```

wc -l counts the selected rows: two lines start with ok.

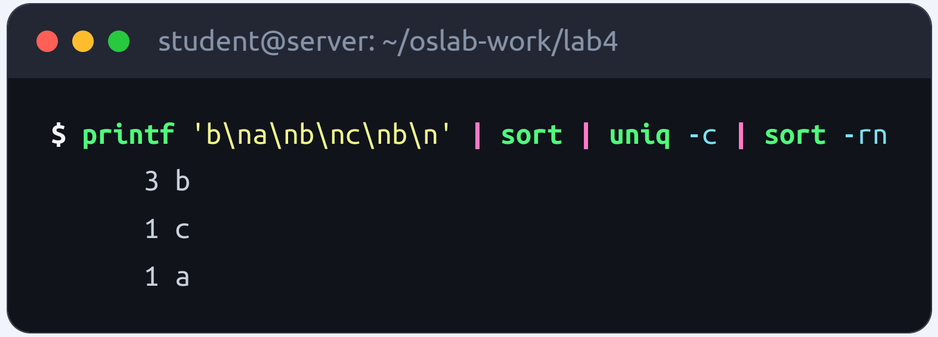
The report needs the sum of the second field of those rows: 3 plus 5.

REMEMBER

Know which quantity you want

Counting rows and adding numbers answer different questions. Check each stage before you trust the last one.

The frequency pipeline



Extract a column, then sort | uniq -c | sort -rn. It counts how often each value occurs and puts the most frequent first.

The first sort puts equal lines next to each other, uniq -c counts them, and sort -rn orders the counts from high to low.

LAB 4

A report you can trust

TechCorp receives small CSV event logs. Count the successes, and keep results apart from diagnostics.

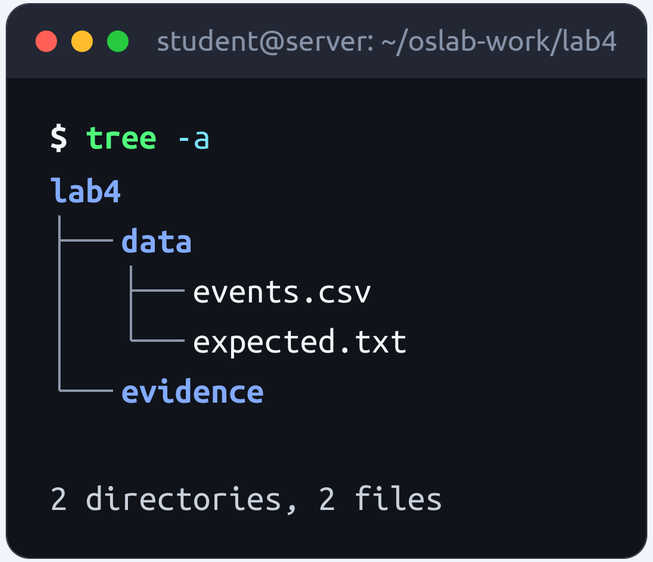
The scenario

SCENARIO

Event logs at TechCorp

Produce a trustworthy success-count report from a small CSV event log, and tell normal output apart from error messages.

Your status word and your numbers are your own. This guide shows example values.



Inside the 120 minutes

Minutes	What happens
0 to 10	Setup
10 to 25	Guided: redirection and pipelines
25 to 35	Prediction
35 to 55	Core 1: write report.sh
55 to 70	Core 2: test your report

Minutes	What happens
70 to 85	Instructor demonstration, Plus and Challenge
85 to 100	Live checkpoint
100 to 110	Debrief
110 to 120	Submit

Six helper commands, and why you use them

Command	What it does
`oslab values lab4`	Shows your status word and the other word. Use them everywhere in this lab
`oslab predict lab4`	Saves your prediction once, before the experiment
`oslab hint lab4 1`	A hint, levels 1 to 3. Hints cost no points
`oslab check lab4`	Runs your report.sh on test files and shows your milestones
`oslab checkpoint lab4`	Opens when the instructor says so, near the end
`oslab debrief lab4`	After the lab: you give the right answers to your prediction questions

Before the lab, run oslab doctor and oslab prelab lab4. It takes about ten minutes.

How you work

Core, Plus, Challenge

Core is for everyone. Plus and Challenge are for students who finish early. Full marks need only Core and the checkpoint.

AI tools

Allowed in Core, Plus and Challenge, so test what they tell you. Not allowed in the prediction and the checkpoint.

You write report.sh

In Core 1 nobody gives you the full script. You build it from the pieces you already know.

A wrong prediction costs nothing. It is marked for being made and corrected, not for being right.

Setup: start the lab

student@server: ~

```
$ export OSLAB_WORKSPACE="${OSLAB_WORKSPACE:-$HOME/oslab-work}"
$ oslab start lab4
$ cd "$OSLAB_WORKSPACE/lab4"
$ mkdir -p evidence
$ oslab values lab4
```

- The first line sets OSLAB_WORKSPACE to ~/oslab-work, unless it is already set. Type it as it is.
- oslab start lab4 makes the lab folder.
- cd takes you into it, and mkdir -p makes the evidence folder.
- oslab values lab4 prints your own words.

Your values: two words that are only yours

The last command prints them

example: your values will be different

```
$ oslab values lab4
Your personal values for lab4 (your-account):
  status   = ready
  other    = done
```

What to do with them

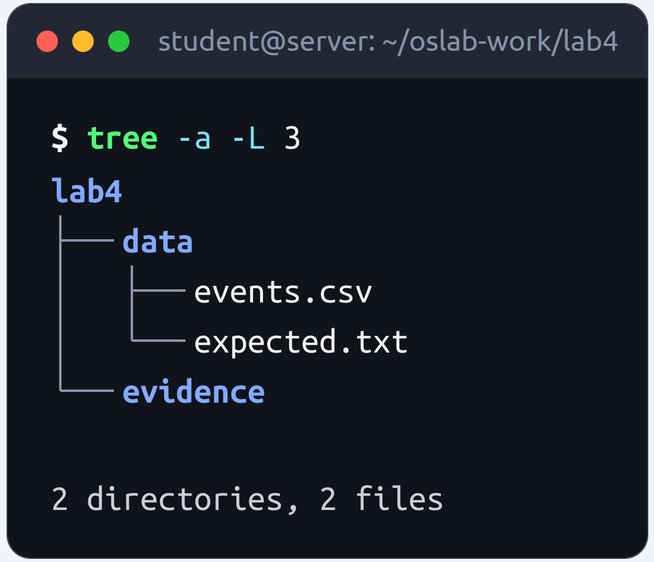
- Write both words down. Your own words are different from the example.
- Use them everywhere in this lab.
- The numbers inside events.csv are also only yours.

The capital words are placeholders

The instruction says	With the example values, you type
`grep '^STATUS,' data/events.csv`	`grep '^ready,' data/events.csv`
`printf 'OTHER,5\n'`	`printf 'done,5\n'`
the label `STATUS total: NUMBER`	the label `ready total: NUMBER`

The instruction cannot know your values, so it writes STATUS and OTHER in capitals where your word belongs. Never type the capital words. Type your own value in their place.

Your folder



data/events.csv has a header and six records. Its numbers are only yours.

data/expected.txt is the report you must produce.

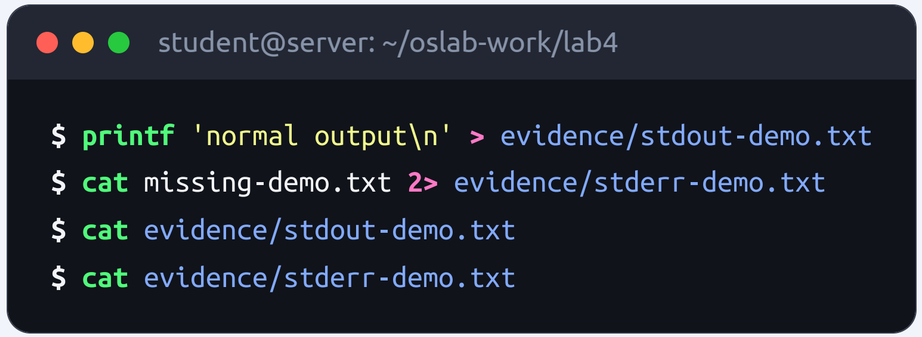
oslab check lab4 now shows TRY on every line, because there is no report.sh yet. The hidden .oslab-managed.json is the helper's. Leave it.

GUIDED

Redirection, then pipelines

Ten to twenty-five minutes, with the instructor.

Step 1: normal output and an error, apart



The second command fails on purpose. > sent stream 1 to the first file. 2> sent stream 2 to the second file.

TRY

Observe

Which file holds the error text? What did the terminal show while the second command ran?

Step 2: both streams into one file, and the status

student@server: ~/oslab-work/lab4

```
$ bash -c 'printf "out\n"; printf "err\n" >&2' > evidence/both.txt 2>&1
$ cat evidence/both.txt
$ cat missing-demo.txt 2> /dev/null
$ echo "status: $?"
```

bash -c runs the quoted text as a small program. The second printf writes to stderr: >&2. 2>&1 sends stderr where stdout goes.

/dev/null throws away what you send to it. The status is not 0, because cat failed.

Step 3: a pipeline, one stage at a time

example values: use your own words

```
$ printf 'ready,3\ndone,2\n' | grep '^ready,'
ready,3
$ printf 'ready,3\ndone,2\n' | grep '^ready,' | cut -d, -f2
3
```

In the instruction these lines say STATUS and OTHER. Type your own words. The example uses ready and done.

Run each line and read the result before you add the next stage. cut -d, -f2 cuts each line at the commas and keeps the second field.

Step 4: look at your data

```
student@server: ~/oslab-work/lab4

$ cat data/events.csv
$ cat data/expected.txt
```

The lab uses a simple two-field CSV without quoted commas.

TRY

Observe

Which rows have your status? Is there a row whose name only starts like your status? What does wc -l count: records or a sum?

PREDICT

Guess first, then try

Before you write the report. Ten minutes.

The prediction

student@server: ~/oslab-work/lab4

```
$ oslab predict lab4
```

It asks three questions and one sentence about your own events.csv. Answer alone, without AI.

Use the pipeline from Step 3 in your head, on your own events.csv.

PREDICTION

The rules

The first answer is the one that counts. A wrong guess costs nothing. You compare it with the real result later, in the debrief.

CORE 1

Write report.sh

Twenty minutes, from minute 35 to 55. Make a script that prints one line you can trust.

What report.sh must do

report.sh must

- Take one argument: the path of a CSV file.
- Select every record whose first field is exactly your status. A look-alike such as STATUS-late must not count.
- Add up field 2 of those records.
- Print the result once: STATUS total: NUMBER, for example ready total: 8. Print STATUS total: 0 when no record matches.
- If the input file does not exist: a message on stderr, nothing on stdout, and a status other than 0.

YOUR TURN

You write it

Nobody gives you the script. Decide where the selection, the field extraction and the sum belong.

Step 1: look at an incomplete pipeline

example value: use your own word

```
$ grep '^ready,' data/events.csv
$ grep '^ready,' data/events.csv | cut -d, -f2
$ grep '^ready,' data/events.csv | cut -d, -f2 | head -n 1
```

Type your own status word where the example says ready. Run the three lines, then compare with your prediction.

TRY

Observe

Which stage first loses information? What does head -n 1 keep, and what does it throw away?

Step 2: two tools you may need

awk adds numbers

student@server: ~/oslab-work/lab4

```
$ printf '2\n4\n' | awk '{ total += $1 } END { print total+0 }'
6
```

+0 makes an empty sum print 0. This demonstration does not solve the lab.

An error on stderr, and a stop

student@server: ~/oslab-work/lab4

```
$ if [ ! -f "$1" ]; then
$     echo "report.sh: no such file: $1" >&2
$     exit 1
$ fi
```

>&2 sends the message to stderr. exit 1 stops with a failing status.

Step 3: your decision

YOUR TURN

Write report.sh in the lab4 folder

Open it with nano report.sh. A staged pipeline and a single awk program are both fine.

If you are stuck

1. oslab hint lab4 1, then 2, then 3.
2. Print each stage of your pipeline.
3. AI is allowed here. Test every suggestion on these small files.

Step 4: run it and compare

student@server: ~/oslab-work/lab4

```
$ bash report.sh data/events.csv > evidence/normal.txt
$ diff -u data/expected.txt evidence/normal.txt
```

No output from diff means the files are the same. oslab check lab4 also runs your script.

TRY

Think

A matching report is useful evidence. Why does matching one file not prove that the script is right for every CSV?

Test your report

Fifteen minutes, from minute 55 to 70. One matching file does not prove the script.

Step 1: make two changed inputs

example values: use your own words

```
$ printf 'status,count\ndone,2\n' > data/no-match.csv
$ cp -- data/events.csv data/changed.csv
$ printf 'ready,7\n' >> data/changed.csv
```

no-match.csv has no row with your status. changed.csv is a copy of the original with one more record, worth 7.

The original file is not changed. >> adds a line at the end.

Step 2: run it on every input, and save everything

student@server: ~/oslab-work/lab4

```
$ {
$   bash report.sh data/no-match.csv
$   bash report.sh data/changed.csv
$   bash report.sh data/missing.csv
$   printf 'missing-input status=%s\n' "$?"
$ } > evidence/edge.txt 2>&1
$ cat evidence/edge.txt
```

Expect your status total 0, then your normal total plus 7, then your error message, then a status that is not 0.

$? is the exit status of the last command. 2>&1 puts the error message in the file too.

What each test contradicts

Case	Expected	A wrong script that it exposes
Your events.csv	The total in expected.txt	Counting rows, or counting a look-alike
No match	STATUS total: 0	Printing nothing, or an error, when nothing matches
Extra records	Your normal total plus 7	Keeping only the first match
Missing input	A message on stderr, and a status that is not 0	Ignoring input errors

grep returns 1 when nothing matches, which is not a read error. State one kind of CSV your script cannot read, for example a quoted comma.

Step 3: check your work

The four Core milestones must show PASS: your own events.csv, no match, extra records and a missing input.

student@server: ~/oslab-work/lab4

```
$ oslab check lab4
```

REPORT

Write down for your report

What does each test contradict? Which test would catch counting rows, using only the first match, counting a look-alike, and ignoring a missing input?

Plus: how many records has each status?

sort puts equal lines next to each other, and uniq -c counts each group.

student@server: ~/oslab-work/lab4

```
$ cut -d, -f1 data/events.csv | sort | uniq -c
```

PLUS

Answer in evidence/plus.txt

Why does the header line status appear in the result?

First watch the instructor show a pipeline in which one stage fails and the pipe still succeeds. Plus and Challenge are not needed for full marks.

Challenge: the same sum without awk

example value: use your own word

```
$ echo $(( $(grep '^ready,' data/events.csv | cut -d, -f2 | paste -sd+ -) ))
```

paste -sd+ - joins the lines with a plus sign, and $(( )) calculates it.

CHALLENGE

Answer in evidence/plus.txt

What does this print for your own data? What happens when no row matches, and why?

LIVE CHECKPOINT

Show what you can do alone

Fifteen minutes near the end of the lab. Worth 3 of the 10 points.

Why is there a checkpoint?

It shows what you can do alone

For most of the lab you may use hints and AI tools. The checkpoint is the one moment without them.

The records are new

The questions use new records, made for you at that moment. An answer copied from a neighbour does not fit.

The instructor sees who needs help

Your answers arrive at once, so a common mistake can be explained before the next lab.

It is fair

Everyone gets the same kind of task, with their own numbers.

What happens, step by step

1. The instructor opens it
Close your AI tools

→

2. oslab checkpoint lab4
Answer the four questions

→

3. The small task
Make the new CSV, run report.sh

→

4. oslab check lab4
A fifth milestone must pass

Before it opens

The command says that the checkpoint is not open yet. Wait for the instructor.

While you work

Use your terminal and what you learned in this lab. No AI tools, and no help from a neighbour.

When you finish

oslab check lab4 shows whether the fifth milestone, checkpoint, passes. You can fix report.sh and check again.

What the checkpoint asks

Part	What you do
Four questions	They use new records that were made for you at this moment. Answer them in the terminal.
The small task	Create the new CSV file the command prints, run your own report.sh on it, and save the output in checkpoint.txt.
The check	Run oslab check lab4. The fifth milestone, checkpoint, must show PASS.

The same report.sh must work on the new file. This is why Core 2 tested more than one input.

DEBRIEF AND SUBMIT

Look back, then hand in

Ten minutes of debrief, ten to submit.

Debrief: look at your prediction again

student@server: ~/oslab-work/lab4

```
$ cat "$OSLAB_WORKSPACE/.records/lab4-predict.json"
$ oslab debrief lab4
```

oslab saved your prediction in a hidden folder, .records. It stays as it was, so the comparison shows which answers you confirmed and which you corrected.

DISCUSS

Class discussion

Why is the number from wc -l a different quantity from the total?

Submit 1: your repository and the copy

student@server: ~

```
$ SUBMISSION_REPO="$HOME/os-gic-YOUR_ID/os-lab-YOUR_ID"
$ mkdir -p "$SUBMISSION_REPO/lab4/evidence"
$ cd "$OSLAB_WORKSPACE/lab4"
$ cp -- report.sh "$SUBMISSION_REPO/lab4/"
$ cp -- evidence/normal.txt evidence/edge.txt "$SUBMISSION_REPO/lab4/evidence/"
```

Replace YOUR_ID with your own ID. The orange text is the part you change. Use the absolute path of your existing course repository, never a neighbour's.

Example: with the ID 0123, the first line becomes SUBMISSION_REPO="$HOME/os-gic-0123/os-lab-0123", and echo "$SUBMISSION_REPO" prints /home/student/os-gic-0123/os-lab-0123.

If you did Plus or Challenge, copy evidence/plus.txt the same way. The course website checks that these exact names are there.

Submit 2: the final tree and Git

student@server: ~/os-gic-YOUR_ID/os-lab-YOUR_ID

```
$ tree
lab4
├── evidence
│   ├── edge.txt
│   └── normal.txt
├── README.md
└── report.sh

1 directory, 4 files
```

1. Write README.md from the lab's template, in your own words.
2. The prediction must have been written before you ran anything. Copying it into the README only records it.
3. git status --short, then add only your lab files, then commit and push.
4. No passwords, personal shell files, generated binaries or disk images.

Submit 3: commit and push

student@server: ~

```
$ cd ~/os-gic-YOUR_ID
$ git pull
$ git add os-lab-YOUR_ID/lab4
$ git commit -m "Lab 4"
$ git push
```

git pull comes first, in case you pushed from your own computer since.

Replace YOUR_ID with your own ID, as before.

How it is marked: 10 points

Evidence for the lab objectives	Points
Report: oslab check lab4 passes the four Core milestones (your own events.csv, no match, extra records, missing input)	2
Tests: normal.txt and edge.txt show the results, and the README says what each test contradicts	2
Prediction saved in time, and corrected with oslab debrief lab4	2
Live checkpoint: the four answers and your report.sh working on the new file	3
Complete submission: the required files in your repository, on time	1

Every row is marked from your oslab records and the files in your repository, so run oslab check lab4 until it shows what you expect. Your prediction earns its first point for being made, not for being right. Plus and Challenge are not needed for full marks.

Common mistakes

Mistake	What you see	Fix
Typing STATUS instead of your own word	No matching rows, so a total of 0	Use the word from oslab values lab4
A pattern without the comma	A look-alike such as STATUS-late is counted	Match the whole first field: ^STATUS,
wc -l after the filter	The number of rows, not the total	Add field 2 with awk
Error text on stdout	A message inside the report line	Send it to stderr with >&2
No exit 1 after the message	A missing file looks like a success	Finish with a status other than 0

Processes, jobs and signals

Optional extensions. They use only your own, bounded processes.

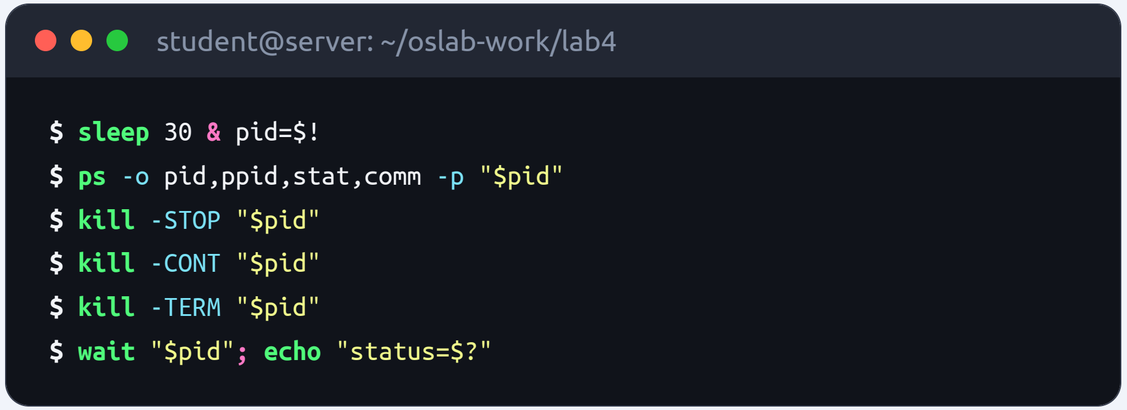
Looking at processes

Command	What it shows	Output
`ps aux`	A snapshot of all processes	Can be redirected
`ps --forest`	The process tree	Can be redirected
`ps -o pid,ppid,stat,comm -p PID`	One process, with its parent and state	Can be redirected
`top, htop`	A live view, updated all the time	Interactive: take a screenshot

Start exactly one process of your own and keep its PID. Do not kill processes by name.

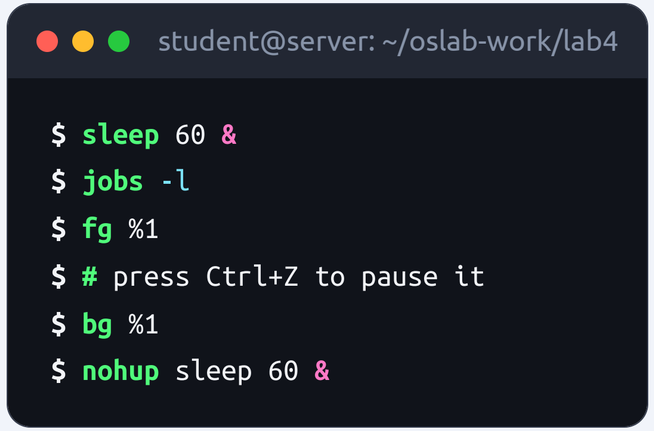
Jobs and signals

Signal	Number	Effect	Can be caught?
SIGTERM	15	Asks the process to end	Yes
SIGKILL	9	Ends it at once	No
SIGSTOP	19	Pauses it	No
SIGCONT	18	Resumes it	Yes
SIGINT	2	Ctrl+C from the keyboard	Yes



$! is the PID of the last background job. A stopped process shows state T. A wait status above 128 means a signal ended it.

Job control



A trailing & runs a command in the background. jobs -l lists your jobs with their PIDs.

fg %1 brings job 1 to the front. Ctrl+Z pauses the front job, and bg %1 lets it run in the background.

nohup keeps a command running after the terminal closes. Use these only on your own, short commands.

Orphans and zombies

	Orphan	Zombie
Who ends first?	The parent	The child
Is the child still running?	Yes	No, it has exited
State in ps	S or R, normal	Z, shown as defunct
Who cleans it up?	The system gives it a new parent, often PID 1	Its parent must call wait
Can you kill it?	Yes, it is a normal process	No, it is already dead: fix the parent

Run these demonstrations for a few seconds, in a disposable environment, and leave nothing running.

Key takeaways

01

Three streams

stdin, stdout and stderr are separate. Redirect each on purpose.

02

Pipes

A pipe carries only stdout. Build a pipeline one stage at a time.

03

Rows are not counts

Check which quantity each stage produces before you trust the total.

04

Test the edges

No match, a changed count and a missing input show what the script really does.

Next session

COMING UP

Lab 5: threads, kernel workers and signals

Look at threads inside a process, the kernel's own workers, and how a signal reaches a process.

SOURCE

Reading

Silberschatz, Galvin and Gagne, *Operating System Concepts*, 10th edition, chapter 3, Processes.